UTTARKUNTH

Master Ecosystem Architecture & AI Studio Context Document

Document version: 2.0
Prepared: 3 November 2026
Official domain: https://uttarkunth.com
Official business email: info@uttarkunth.com
Parent brand: Uttarkunth
Product scope: Stays • Products • Productions • Community
Motto: *Learn by Doing • Grow by Serving* 💜

Purpose of this document
This is the master context and architecture brief to accompany the Uttarkunth source code when it is reviewed or continued in Google AI Studio or another AI development environment. It describes the intended business, product, application, backend, security, deployment and roadmap architecture. It also records the latest implementation and production-state claims supplied by the owner from Antigravity's 3 November 2026 handoff report.

Important status distinction: This document is not a live repository audit, live Supabase audit, penetration test or deployment certificate. Build and test results below are reported by Antigravity and have not been independently rerun here. Where production database records or deployment configuration are involved, verify them against the actual project before treating them as current facts.

Contents

1. Executive summary
2. Brand, purpose and operating philosophy
3. Business ecosystem and four product pillars
4. Users, roles and shared account model
5. Application architecture and technology stack
6. Route and information architecture
7. Supabase data architecture and migration history
8. Security, permissions and transaction invariants
9. Core user and business workflows
10. Latest reported implementation inventory
11. Production state and known release blockers
12. Cloudflare, Git and environment configuration
13. Development roadmap and release gates
14. Google AI Studio working instructions
15. Final owner checklist
16. Glossary and source-of-truth rules

---

1. Executive summary

Uttarkunth is a parent brand and a connected digital ecosystem rooted in mountain communities and local enterprise. The current website architecture brings four pillars into one platform:

1. **Stays** - accommodation discovery, property details, room availability, request-based bookings and host operations.
2. **Products** - a future marketplace for authentic Himalayan goods, with seller onboarding, product curation and interest/waitlist pathways.
3. **Productions** - documentaries, films, creative collaborations and filmmaker submissions.
4. **Community** - the Karmayog and Uttarkunth Living University vision, field initiatives, participation, oral-history and community storytelling pathways.

All pillars are intended to share one Uttarkunth identity, public navigation, search experience and account area. Privileged administration remains separate from ordinary user identity and must be granted explicitly.

The reported technology foundation is **Next.js 14 App Router, React 18, TypeScript, Supabase and Cloudflare Workers using OpenNext**. Supabase provides PostgreSQL, authentication, row-level security, SQL

functions/RPCs, triggers, constraints and storage. Cloudflare Workers Builds is intended to build and deploy the site from the owner's Git repository after the owner pushes code.

Current release position

Antigravity's latest report dated 3 November 2026 describes a large amount of application work as complete and reports:

- 101/101 automated tests passed.
- 7/7 isolated PostgreSQL WASM validation stages passed.
- TypeScript typecheck passed with zero errors.
- ESLint passed with zero errors.
- Next.js production build passed with 71 routes.
- OpenNext Worker build produced `.open-next/worker.js` and `.open-next/assets`.
- The production Supabase database was inspected read-only during this final handoff; the report says no remote DDL/DML was executed.

However, the same report explicitly says the latest Productions/Community migration is **pending and has not been applied to production**, the production `platform_admins` table has zero rows, the two canonical properties have no assigned `host_id`, and there are no sellers, products, product interests, legal documents or active host agreements. Therefore, the platform should be treated as a **code-complete release candidate with production activation dependencies**, not as proof that every pillar is fully operational for real users.

---

2. Brand, purpose and operating philosophy

2.1 Parent brand

- **Brand name:** Uttarkunth (use this spelling consistently).
- **Founder:** Gaurav Negi.
- **Official domain:** `uttarkunth.com`.
- **Official business email:** `info@uttarkunth.com`.
- **Brand motto:** "Learn by Doing • Grow by Serving 💜".
- **Long-term learning vision:** Uttarkunth Living University.
- **Mindset:** Karmayog - learn through action, contribution and service rather than focusing only on outcomes.

Uttarkunth is intended as a parent brand and business ecosystem, **not an NGO identity**. Community development and village initiatives are part of the philosophy and future operating model, but the platform must not imply charitable registration, tax exemption, formal university recognition, or government affiliation unless those statuses are independently established.

2.2 Operating philosophy

The guiding principles are:

- Stay connected to local roots, people, knowledge, culture and landscapes.
- Build through practical work and collaboration: "think 5%, do 95% work".
- Reinvest in sustainable business growth and, when formally approved and financially possible, village-development work.
- Be transparent about what is already operating versus what is a proposal or future initiative.
- Avoid making a fixed community-profit allocation promise until the company structure, legal terms and financial policy are formally approved.

2.3 Current operating context

The business began with hospitality and food activity around Jari, Kullu, Himachal Pradesh. The digital platform is broader than one homestay: it is intended to connect local stays, food and goods, creative production and community participation over time.

The café/food operation may be described in brand storytelling or local experience content, but the latest website architecture is organized around the four pillars above. Do not add a separate café POS or restaurant-management platform to the current scope unless the owner requests it.

2.4 Brand and visual system

Latest recorded design direction:

Token	Value	Intended use
Forest green	#263D32	Primary brand colour
Terracotta	#C55B23	Accent / action colour
Warm background	#F5F0E6	Main warm surface
Beige	#D9CCB8	Secondary surface / borders
Dark text	#29251F	Primary text

Typography direction: **Cormorant Garamond** for expressive headings and **Manrope** for interface/body text, subject to actual implementation in the repository.

Preserve the owner's official logo and existing media assets. Do not replace the real logo, property images, approved posters or the existing hero video with generated substitutes. Known hero line: **"Ideas born in the mountains. Built through people."**

---

3. Business ecosystem and four product pillars

3.1 Pillar I - Stays

Purpose

A curated discovery and request-booking platform for mountain homestays and retreats, initially focused on the Himalayan region. The intended geography includes Himachal Pradesh, Jammu & Kashmir and Uttarakhand, with authentic property supply rather than fabricated catalogue inventory.

Core features

- Public stays discovery and property detail pages.
- Destination pages and destination discovery.
- Property photos, descriptions, amenities, house rules and room inventory.
- Room-level availability and unavailable-date blocks.
- Request-only booking flow; a request is not an instant confirmed reservation.
- Server-side price estimate and booking validation.
- Guest trips/status view.
- Host workspace for authorised hosts to manage property data, rates, availability and booking requests.
- Host partnership/onboarding page.
- WhatsApp enquiry as a communication channel only, not as booking confirmation.

Canonical properties reported

Property	Location	Reported detail	Important note
Yash Home Stay	Jari, Kullu, Himachal Pradesh	Existing canonical property; smoke-test report mentions ₹900 as a room/base value	Verify the current live rate and room configuration in Supabase before publishing. host_id is reported as NULL.
Wonderway Retreat	Jari, Kullu, Himachal Pradesh	"Mountain View Room with Kitchenette"; max occupancy 2; smoke-test report mentions ₹3,500	Verify the current live rate and room configuration in Supabase before publishing. host_id is reported as NULL.

Jari is positioned as a quieter village near Kasol, approximately 9 km before Kasol according to owner-provided context. Yash Home Stay is described as a short walk from the Jari bus stand. Confirm transport, distance and amenity claims before using them as live structured data.

Commercial rules currently reported

- Booking model: request-based.
- Payment: guest pays the host directly at the property.
- Online payment gateway: not enabled.
- Production default commission: 0% in the reported current state.
- No active host agreement or commission contract exists in the reported production database.

A previous business concept considered an introductory 0% commission period followed by a later commission. That is **not an active contractual setting**. Never hardcode a future commission or treat it as agreed until an authorised agreement is reviewed, accepted and recorded.

3.2 Pillar II - Products

Purpose

A curated Himalayan goods marketplace intended to connect regional makers, artisans and responsible producers with customers.

Reported application features

- Nine product/category groupings are represented in the UI/content layer.
- Public products catalogue and product detail pathways.
- Product-interest/batch waitlist form.
- Seller/maker application page.
- Seller workspace.
- Administrative seller and product moderation desk.
- Product provenance/curation concepts.

Current status

Antigravity reports the database tables and moderation structures were added in Phase 2F, but the production counts are zero for sellers, products and product interests. Therefore, the public catalogue should remain **Coming Soon / preview** until real sellers and products have been onboarded and approved. Do not invent seller names, stock, product availability, provenance, prices, shipping options or customer reviews.

No checkout or online payments are part of the currently authorised scope.

3.3 Pillar III - Productions

Purpose

An Uttarkunth film and documentary space for mountain stories, culture, local knowledge and creative collaboration.

Named titles in the current brief

- **Aaj Ka Devta**
- **Ruk Gayi Gupt Bali**

Reported application features

- Productions hub and title detail routes.
- Video-player container.
- Credits, participant consent and ethics disclosures.
- Filmmaker/collaborator application form.
- Project proposal/pitch desk with creative-role and project-detail fields.
- Authenticated creator workspace showing submitted pitches and review status.
- Administrative production moderation with approve, request-revision and decline pathways.
- Member-access/streaming-pass concepts are mentioned in the report.

Current status and verification needs

The code/report inventory indicates these interfaces and API routes were added. The pending Phase 2G/2H migration includes production-related tables, so production database-backed submissions and moderation cannot be considered live until that migration is separately reviewed and authorised. Confirm actual video source files/URLs, playback, access control and mobile behaviour. Static generation of a film page does not prove that the film itself streams or that member-only access is enforced.

3.4 Pillar IV - Community and Karmayog

Purpose

Present Uttarkunth's learning-by-doing philosophy and create transparent pathways for participation in local research, culture, environment, skills and village initiatives.

Reported application features

- Community philosophy/vision hub.
- Nine initiative cards and initiative detail pages.
- Participation/fellowship application pathway.
- Living oral-history and story archive.
- Story submission with participant consent.
- User-facing participation status area.
- Admin moderation for applications and stories.

Nine initiative concepts listed in the report

1. alpine-ecology-trails - Alpine Ecology & Zero-Plastic Trails (Parvati Valley).
2. kath-kuni-conservation - Kath-Kuni Vernacular Architecture Preservation (Kullu & Naggar).
3. youth-digital-mentorship - Youth Digital & Creative Mentorship (Jari Hub).
4. indigenous-seed-bank - High-Altitude Seed Bank & Agroecology (Spiti & Upper Kullu).
5. oral-history-archive - Himalayan Oral Folk Songs & Living Archives (Parvati & Seraj).
6. wool-handloom-revival - Traditional Wool, Weaving & Natural Dye Revival (Kullu & Kinnaur).
7. homestay-waste-stewardship - Community Homestay Waste & Energy Management (Jari).
8. flora-forest-documentation - Mountain Forest & Medicinal Flora Documentation (Great Himalayan National Park buffer area).
9. pastoralist-route-archiving - Transhumant Pastoralist Rights & Route Archiving (High Passes).

Status rule: These are reported as initiative concepts/pages. Do not describe them as active, funded, staffed, completed or officially partnered field programmes unless the owner supplies evidence that the work is actually happening. The pending database migration also means community submissions/moderation require production-schema activation before being treated as live workflows.

---

4. Users, roles and shared account model

The intended product has one common Supabase Auth identity. One person may use different areas of Uttarkunth over time, but access to each role-specific workspace must be checked independently.

User type	Typical capabilities	Access principle
Visitor	Browse public pages, stays, films, products preview, initiatives and search	Public read access only to published/approved material
Guest	Profile, booking requests, trips, notification/status views	Must be authenticated for private booking/account data
Host / co-host	Manage assigned property, rates, availability and stay requests	Explicit property assignment and role scope required
Seller / maker	Apply, manage own seller profile/products and enquiries	Seller onboarding and moderation required
Filmmaker / collaborator	Submit profile and project proposal; view own submission status	Authenticated identity and ownership-scoped access
Community participant	Submit participation applications/stories and view own status	Authenticated where required; explicit consent for stories/media
Platform administrator	Moderate platform resources and perform governance actions	Explicit allowlisted administrator record; never inferred from email or user metadata

Admin identity rule

The production report says public.platform_admins contains zero records. No administrator is automatically provisioned. An email address, including a founder or official business email, must not confer admin authority by itself. Admin bootstrap is a deliberate owner-controlled security operation.

Host assignment rule

The latest report says Yash Home Stay and Wonderway Retreat both have `host_id = NULL`. This means host-specific operational workflows are blocked until the owner explicitly assigns an authorised host or signatory. Do not infer property ownership from the user's email address or profile metadata.

---

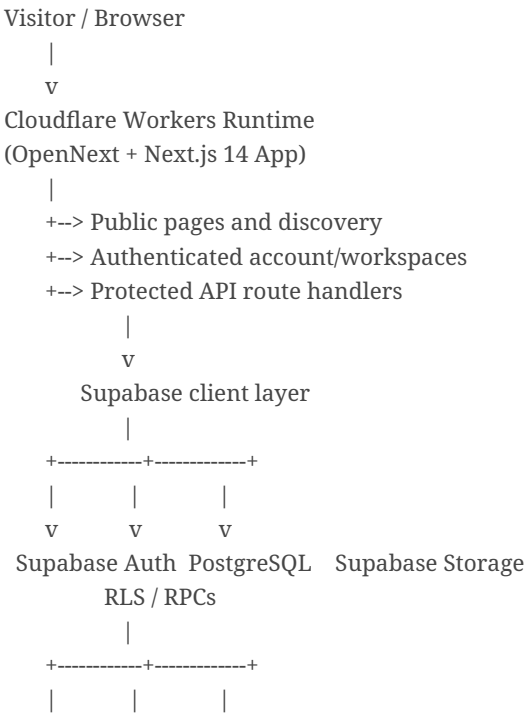
5. Application architecture and technology stack

5.1 Reported stack

Layer	Technology / service	Responsibility
Web application	Next.js 14 App Router	Routes, server-rendered pages, static generation and application structure
UI	React 18, TypeScript	Components, interactive forms and typed application logic
Styling	Tailwind CSS and shared design tokens	Responsive visual system
Icons	Lucide icon set	Interface iconography
Validation	Zod	Input validation at UI/API boundaries
Backend database	Supabase PostgreSQL 17.11 (reported)	Persistent relational data, constraints, views and SQL functions
Identity	Supabase Auth	User identity and session management
Data security	PostgreSQL RLS, grants, triggers and RPCs	Server/database-enforced permissions and business invariants
Media	Supabase Storage (reported property-photo bucket)	Property images and other approved media
Deployment runtime	Cloudflare Workers	Edge runtime for the Next.js application
Next.js adapter	OpenNext for Cloudflare	Packages Next.js output for Workers
Source control	GitHub (owner-controlled)	Versioned source and release history
Build/deploy automation	Cloudflare Workers Builds	Builds and deploys after owner pushes to connected branch

Reported Supabase project label: `uttarkunth-stays`.
Reported project reference: `xyplgugxpmlnjsvlosdg`.
Reported database region: `ap-northeast-2` (Seoul).
These values are recorded from the supplied handoff report and should be confirmed in the relevant dashboards before operational changes.

5.2 High-level architecture



v v v
Stays Products Productions/Community

The diagram represents the intended logical architecture, not a measured network trace. Verify the actual server/client boundaries and Supabase client usage in source code.

5.3 Request and trust boundaries

Recommended request flow for protected operations:

1. Browser submits a request to a server route handler or trusted server action.
2. Server validates the session and parses input using the relevant schema.
3. Server derives identity and role from the authenticated session and database, not from client-provided role fields.
4. Server calls a narrowly scoped database query or RPC.
5. PostgreSQL RLS, grants, function checks, constraints and triggers enforce the final security boundary.
6. Server returns only the fields the caller is entitled to see.

Never rely on hiding buttons, hiding routes or client-side checks as the sole security control.

5.4 Rendering strategy

The report says the application built 71 routes and statically generated selected production/community pages. Use static generation only for content safe to publish publicly. Private account data, booking status, moderation data, submissions and other user-specific information must be fetched through authenticated, permission-checked paths and must not leak into public static output or caches.

---

6. Route and information architecture

The routes below combine the latest Antigravity inventory with the established site architecture. The actual route tree in the repository is authoritative; this is a functional map, not a claim that every route has been independently opened and tested.

Area	Route examples	Purpose
Core	/, /about, /contact	Homepage, story, contact
Stays	/stays, /stays/[slug], /stays/destinations/[slug], /stays/partner	Discovery, property detail, destination pages, host partnership
Products	/products, /products/[slug], /products/sell	Catalogue, product detail, seller application
Seller workspace	/account/products/seller	Seller tools (permission-gated)
Productions	/production, /production/[slug], /production/collaborate	Film hub, film detail, filmmaker collaboration
Creator workspace	/account/productions	Own pitches, submission status and access information
Community	/community, /community/initiatives, /community/initiatives/[slug], /community/participate, /community/stories	Philosophy, initiative pages, participation and stories
Community account	/account/community	Own participation and submission status
Identity/account	/login, /signup, /account, /account/profile, /account/settings	Authentication and profile settings
Guest	/trips and/or account trip routes	Guest booking requests and trip status; verify canonical current path
Search	/search	Cross-pillar search
Notifications	/account/notifications	User activity and notification hub
Host	/host and/or /account/stays/host	Host tools; verify actual canonical route
Administration	/admin, /admin/products, /admin/productions, /admin/community	Protected governance and moderation

Navigation principles

- Public visitors should be able to discover the four pillars without signing in.
- Authentication should be requested only when a user enters a private workflow.
- A public route must not reveal private records or administrative functions.
- Legacy URLs may require redirects where they have already been published or indexed.
- Mobile navigation and search must remain usable and must not expose privileged navigation to unauthorised users.

7. Supabase data architecture and migration history

7.1 Known data domains

The following objects are reported across prior project context and the latest handoff. This is not a complete schema dump. Always inspect the actual migrations, remote schema, foreign keys, indexes, policies, grants, triggers, functions and generated TypeScript types before changing the database.

Domain	Reported tables / objects	Purpose
Identity/access	auth.users, application profiles, platform_admins	User identity, profile data and explicit admin authority
Stay inventory	properties, room_units, destinations	Approved property listings, rooms and destinations
Booking	booking_requests, stay_bookings, room_availability_blocks	Guest requests, booking records and unavailable dates
Availability	public_room_availability view; availability-block RPCs	Public availability display and controlled host/admin date blocking
Governance	admin_audit_log, legal_terms_documents, host_agreements	Audit, legal document versions and agreement lifecycle
Products	sellers, products, product_interests, rate_limits	Seller profiles, product catalogue, interest submissions and rate limiting
Production/community (pending)	filmmaker_profiles, production_proposals, community_initiatives, community_participations, community_stories, user_notifications	Creator, proposal, initiative, participation, story and notification data
Media	Supabase Storage buckets including reported property-photos	Property and approved platform media

7.2 Reported production migration history

The latest handoff says the remote `supabase_migrations.schema_migrations` table contains exactly these three authorised versions:

Version	Reported migration name	Status in latest report
20261103114500	phase2d_security_remediation	Applied / recorded remotely
20261103114530	phase2e_security_remediation	Applied / recorded remotely
20261103114600	phase2f_products_marketplace	Applied / recorded remotely

Do not reapply these migrations. Reconcile local migration files against remote migration history. Do not edit or delete historical applied migrations to make local history appear clean. If local migration files differ from the applied SQL, document the difference and create a new reviewed corrective migration only after explicit approval.

7.3 Reported local pending migration

The latest report names this local file:

`supabase/migrations/20261103140000_phase2g_2h_productions_community_pending_approval.sql`

It is reported as pending owner production authorisation and not executed remotely. It defines or supports six tables:

- filmmaker_profiles
- production_proposals
- community_initiatives
- community_participations
- community_stories
- user_notifications

The report says it was validated using an isolated PostgreSQL WASM/PGlite environment. That validates the migration within that isolated test environment; it does **not** mean the production Supabase database has those tables. Do not apply the migration automatically, as part of the application build, or during Cloudflare deployment.

7.4 Reported Phase 2D and 2E objects

The Antigravity summaries describe the following security/governance additions. Confirm exact function signatures and names before using them:

- is_platform_admin() and is_platform_admin_for_user() for explicit admin checks.
- Controlled availability-block operations, including set_room_availability_block() and delete_room_availability_block().
- A secure booking-decision RPC with transaction/locking controls. Earlier project reports used differing names; inspect the live function list and source before relying on a specific name.
- public_room_availability view.
- admin_audit_log intended as append-only audit history.
- legal_terms_documents table and host_agreements lifecycle with digest/immutability controls.
- terminate_host_agreement() function.

7.5 Reported Phase 2F objects

- sellers
- products
- product_interests
- rate_limits
- check_rate_limit()
- Product moderation lifecycle trigger and RLS policies.

The latest reported production counts for sellers, products and product interests are all zero.

7.6 Reported production database counts/state

Item	Latest report value	Interpretation
platform_admins	0	No administrator has been provisioned
host_agreements	0	No recorded host agreements
legal_terms_documents	0	No approved legal document versions registered
sellers	0	No seller accounts/records in the reported count
products	0	No products in the reported count
product_interests	0	No product-interest submissions in the reported count
Properties with host assignment	0	No assigned host in the reported count
Yash Home Stay host_id	NULL	Host management requires explicit assignment
Wonderway Retreat host_id	NULL	Host management requires explicit assignment

These are owner-supplied Antigravity report values, not a fresh query executed during preparation of this document.

---

8. Security, permissions and transaction invariants

8.1 Authentication and authorisation

- Supabase Auth is the identity provider in the reported architecture.
- Every private record must be scoped to its authenticated owner or authorised role.
- User-supplied IDs, roles, ownership claims, prices or moderation states must not be trusted without server/database validation.
- Admin status must be stored/verified through the dedicated admin-authorisation mechanism, not inferred from email, user metadata or frontend state.
- Host, seller, filmmaker and participant capabilities must be scoped to the corresponding resources and status.

8.2 Row Level Security

RLS is a baseline security boundary for exposed tables. For every table, review:

- RLS enabled status.
- SELECT, INSERT, UPDATE and DELETE policies.
- Table-level grants and function execute permissions.
- Whether policies use a trusted identity/ownership helper.
- Whether service-role use is restricted to server-only code.
- Whether storage policies match the intended public/private media use.

A UI route being hidden is not a replacement for RLS or server-side authorisation.

8.3 Booking concurrency and integrity

The reported backend includes concurrency protection for room/date conflicts and a no-overlap rule for confirmed reservations. The desired invariant is:

- A room cannot have overlapping confirmed stays.
- Check-in/check-out date ranges should be modelled consistently; half-open intervals [check-in, check-out) allow a new stay to begin on another guest's checkout date.
- Booking decisions must revalidate room capacity, dates, property status and availability inside the transaction that changes the booking state.
- Concurrent requests must not bypass availability checks.
- Once confirmed, important booking/price snapshots must not be silently rewritten.

Verify the exact exclusion constraint, index operators and RPC transaction logic in the actual SQL before treating these invariants as proven.

8.4 Auditability and rate limiting

- Privileged moderation and governance actions should produce append-only audit entries.
- Audit history must not be editable/deletable by ordinary users or application paths.
- Public forms should have server-side validation, abuse/rate limiting and safe error handling.
- Rate limits must be applied to the appropriate identity/IP/resource dimensions without exposing private information.

8.5 Secrets and sensitive configuration

Never commit or expose:

- Supabase service-role key.
- Database password or connection string containing credentials.
- Cloudflare API tokens or account secrets.
- Private signing keys, OAuth secrets or production session secrets.
- Local .env, .env.local, .dev.vars or similar secret files.

The Supabase anon/publishable key is designed for client use only when RLS is correctly configured. The service-role key bypasses normal RLS and must remain server-side and tightly controlled.

---

9. Core user and business workflows

9.1 Guest booking journey

1. Visitor browses published/approved stays.
2. Visitor opens a property and reviews room details, house rules and amenities.
3. Guest selects room, dates and party size.
4. Server validates date range, occupancy, room state and current property/room configuration.
5. Server calculates the estimate; do not trust a browser-supplied total.
6. Authenticated guest submits a booking request.
7. Host or authorised administrator reviews the request.
8. A trusted database operation rechecks availability and concurrency before confirming or declining.
9. Guest sees the updated state in their trip/account area.
10. Guest pays the host directly at the property under the current reported payment model.

A submitted request is not a confirmed reservation. WhatsApp is for enquiry/follow-up, not a substitute for a recorded booking state.

9.2 Host workflow

1. Owner explicitly assigns a host or co-host to a property.
2. Host signs in and sees only assigned property resources.
3. Host updates permitted room details, rates and availability.
4. Host reviews incoming booking requests.
5. Host confirms/declines through the secure decision path.
6. Booking status and audit information are retained.

Current blocker: both canonical properties were reported with `host_id = NULL`.

9.3 Product/seller workflow

1. Maker submits seller application.
2. Application is validated and stored through an authorised server/API path.
3. Administrator reviews the seller and verifies relevant provenance information.
4. Seller creates/submits products.
5. Administrator moderates product data and publication status.
6. Approved products become publicly visible.
7. Customer may submit an interest/waitlist enquiry.

Current blocker: production counts for sellers, products and product interests are reported as zero. Keep the catalogue in preview/Coming Soon state until real inventory is approved.

9.4 Production collaboration workflow

1. Creator signs in and submits a collaborator profile or film proposal.
2. Server validates role, content and required consent.
3. Submission is stored with an owner-scoped relationship.
4. Creator views their own submission and review state.
5. Administrator reviews, approves, requests revision or declines.
6. Audit trail records the moderation action.

Current blocker: required production/community tables are included in the reported pending migration and are not yet verified in the production database.

9.5 Community participation and story workflow

1. Visitor reads the initiative and its real current status.
2. Participant submits an application or story with clear consent.
3. Server validates and records the submission.
4. Participant sees their own status.
5. Administrator reviews the application/story.
6. Only approved stories are published; consent and attribution conditions are respected.

Current blockers: pending migration, no verified live participation records and no evidence that all nine initiatives are active field programmes.

---

10. Latest reported implementation inventory

The inventory below is transcribed and organised from the owner's latest Antigravity handoff. It records what Antigravity says was changed; it is not an independently verified source-code diff.

10.1 Productions and Community foundation

File / path	Reported change
supabase/migrations/20261103140000_phase2g_2h_productions_community_pending_approval.sql	New local pending migration for six production/community tables and associated security policies/privileges
lib/constants.ts	Added community initiative metadata, oral-history/story content and types

File / path	Reported change
lib/validations.ts	Added Zod schemas for filmmaker, proposal, participation and story submission inputs
lib/supabase/types.ts	Added TypeScript definitions for six pending Phase 2G/2H tables
app/production/collaborate/page.tsx	Updated co-production pitch desk
components/filmmaker-collaborate-form.tsx	New filmmaker profile and project proposal form
app/api/production/apply/route.ts	New authenticated filmmaker profile API
app/api/production/proposals/route.ts	New authenticated proposal submit/fetch API
app/account/productions/page.tsx	Updated creator workspace
app/admin/productions/page.tsx	New production moderation page
components/productions-moderation-client.tsx	New moderation UI
app/api/admin/productions/route.ts	New protected moderation API with reported audit logging

10.2 Community features

File / path	Reported change
app/community/page.tsx	Expanded Community/Karmayog hub
app/community/initiatives/page.tsx	New initiatives hub
app/community/initiatives/[slug]/page.tsx	New initiative detail route with SSG
app/community/participate/page.tsx	New participation/fellowship form route
components/community-participation-form.tsx	New participation form component
app/api/community/participate/route.ts	New participation API route
app/community/stories/page.tsx	New oral-history/story archive page
components/community-story-form.tsx	New story contribution form with consent checkbox/flow
app/api/community/stories/route.ts	New story fetch/submission API route
app/account/community/page.tsx	Updated participant workspace
app/admin/community/page.tsx	New community moderation page
components/community-moderation-client.tsx	New community moderation UI
app/api/admin/community/route.ts	New protected community moderation API

10.3 Cross-pillar integration and project configuration

File / path	Reported change
app/search/page.tsx	New universal search page
components/site-wide-search.tsx	New cross-pillar search component
app/account/notifications/page.tsx	New activity/notifications page
app/api/account/notifications/route.ts	New aggregated activity API route
app/account/page.tsx	Updated shared account hub
components/header.tsx	Updated navigation/search/account links
components/admin-nav.tsx	Added Productions and Community admin navigation
tests/phase2g-2h-ecosystem.test.ts	New Phase 2G/2H ecosystem tests
package.json	Updated npm test script to include new tests
scripts/test-isolated-pg.mjs	Added Stage 7 migration validation
DEPLOYMENT.md	Updated OpenNext/Cloudflare deployment documentation

This list may not include every changed file. Before Git upload, obtain a real `git status --short`, diff summary and tracked/untracked file inventory from the actual workspace.

---

11. Production state and known release blockers

11.1 What is reported as complete locally

Check	Antigravity-reported result	Confidence note
Isolated PostgreSQL WASM	7/7 stages passed	Reported; rerun from final workspace
Automated tests	101/101 passed	Reported; rerun from final workspace
TypeScript	npx tsc --noEmit passed, 0 errors	Reported; rerun
ESLint	npm run lint passed, 0 errors	Reported; rerun
Next.js build	npm run build passed, 71 routes	Reported; rerun and inspect output
OpenNext Worker build	npm run build:worker passed; .open-next/worker.js produced	Reported; rerun from clean workspace and confirm assets
Git status	Earlier status check exited code 1	Unresolved; root cause not explained in

Check	Antigravity-reported result	Confidence note
		handoff
Production Supabase mutations	None during final read-only inspection	Reported by Antigravity; this document made no remote calls

11.2 What is not ready to be called fully operational

1. **Production/Community schema:** local Phase 2G/2H migration remains pending. Database-backed forms and dashboards must be marked unavailable or safely gated until a reviewed migration is authorised and applied.
2. **Admin operations:** platform_admins count is reported as zero. Admin pages may exist, but there is no provisioned production administrator to use them.
3. **Host operations:** Yash Home Stay and Wonderway Retreat have host_id = NULL in the report. Host approval/management is not operational until authorised assignment.
4. **Legal documents:** legal_terms_documents is reported empty. Do not represent draft/empty documents as accepted legal terms.
5. **Host agreements:** no active or recorded host agreements are reported. Do not activate commission or payout terms.
6. **Products:** no sellers, products or interests are reported. Marketplace remains a preview until real data is added and approved.
7. **Video delivery/access:** actual media URLs, stream playback and member-only access need browser/device testing; a generated page is not proof of playback.
8. **Community field operations:** the nine initiative pages are concepts unless real staffing, activity, partners, funding and outcomes are documented.
9. **Git workspace:** the failed Git status command needs diagnosis before staging or initialising a repository.
10. **Cloudflare settings:** the actual wrangler.jsonc, package.json, open-next.config.ts, .gitignore and .env.example must be inspected before owner push.

11.3 Important distinction: development versus launch

Development, local builds and controlled staging tests can proceed before the business has finalised every commercial/legal document. A public operating platform that collects personal information, accepts applications, facilitates bookings or conducts marketplace transactions needs accurate privacy, terms, cancellation, host/seller and consent notices appropriate to the real service. Obtain qualified Indian legal/accounting advice for applicable entity, tax, consumer, privacy, accommodation and other operational obligations. This document is not legal advice and does not claim that any government registration or approval has been obtained.

---

12. Cloudflare, Git and environment configuration

12.1 Intended release flow

```
Owner reviews local project
        |
        v
Antigravity / development environment verifies code
        |
        v
Owner checks migration status, Git diff and secret exclusions
        |
        v
Owner commits and pushes to the GitHub repository
        |
        v
Cloudflare Workers Builds runs the configured build/deploy pipeline
        |
        v
uttarkunth.com serves the deployed Worker
```

The owner controls Git upload/push. Cloudflare is expected to deploy through its connected Git integration. An AI agent must not independently push code or deploy to production.

12.2 OpenNext output

Reported command:

`npm run build:worker`

Expected generated output:

`.open-next/worker.js`
`.open-next/assets/`

The actual `wrangler.jsonc` must define the Worker entry point and static assets directory correctly. `.open-next/assets` is a generated static asset directory, not itself a Cloudflare deploy command.

Cloudflare Workers Builds has a build stage and a deploy stage. For the existing OpenNext setup, the expected commands commonly take this shape:

- Build command: `npm run build:worker` (only if it builds and does not deploy).
- Deploy command: `npx wrangler deploy` (used by Cloudflare's pipeline after the build).

Confirm the project's actual OpenNext/Wrangler configuration and current Cloudflare dashboard options before saving production settings. Do not use a separate arbitrary output-directory value in place of the Worker configuration.

12.3 Configuration files to inspect

- `wrangler.jsonc` - Worker name, `main` entry point, assets binding/directory, compatibility date and flags.
- `package.json` - scripts, exact Next/OpenNext/Wrangler versions and whether build scripts have deploy side effects.
- `open-next.config.ts` - adapter configuration.
- `.gitignore` - secret/local/generated file exclusions.
- `.env.example` - safe variable names and placeholders only.
- `DEPLOYMENT.md` - must match the real configuration.

The report mentions `nodejs_compat` and a compatibility date of `2024-12-30` or newer. Verify the exact value in the current project; do not assume it remains correct simply because it appears in a prior report.

12.4 Environment variables

Variable	Handling principle
NEXT_PUBLIC_SUPABASE_URL	Public project URL; safe to expose as a public configuration value
NEXT_PUBLIC_SUPABASE_ANON_KEY	Publishable/anon key intended for client use only with correctly configured RLS
SUPABASE_SERVICE_ROLE_KEY	Highly privileged secret; server-side only; configure as encrypted runtime secret; never prefix with NEXT_PUBLIC_
ALLOWED_ORIGINS	Server/runtime configuration; verify how the application parses and enforces it

Inspect actual code to identify which variables are needed at build time and which are required at runtime. Do not place a service-role key in a public build variable, browser bundle, source file, log or `.env.example` with a real value.

12.5 Git safety

The latest report says a Git status check exited with code 1 and did not provide the cause. Before any staging:

1. Check whether the workspace already contains `.git`.
2. Confirm the current working tree root, branch and remotes.
3. Run `git status --short` and inspect the result.
4. Review `.gitignore` and scan for environment/credential files.
5. Inspect diffs and untracked files.
6. Exclude `node_modules`, local caches, private environment files, logs, test database artefacts and unnecessary generated output.
7. Do not initialise a repository or replace a remote until you know the current repository state.

The owner should review the exact files before staging. Do not blindly run `git add .` when the repository state and ignore rules have not been checked.

13. Development roadmap and release gates

This roadmap prioritises reliable production behaviour over adding more visible pages.

Gate 0 - Release workspace integrity

- Confirm the actual project directory and repository identity.
- Diagnose the Git status exit-code 1.
- Generate exact changed/new/deleted file inventory.
- Review .gitignore, .env.example and secret exposure.
- Inspect the final Cloudflare/OpenNext configuration.
- Re-run all reported checks from the final code state.

Exit condition: Owner can review the exact source diff and confidently upload code without leaking secrets or overwriting the wrong repository.

Gate 1 - Database migration reconciliation

- Confirm the three reported Phase 2D/2E/2F versions exist once in remote migration history.
- Compare local migration files and checksums/SQL against the actually applied definitions.
- Preserve historical applied migrations unchanged.
- Review the pending Phase 2G/2H migration as a separate owner-authorised production change.
- Confirm table ownership, grants, RLS policies, indexes, foreign keys, triggers, functions and rollback plan.
- Apply only after approval, using a controlled migration workflow separate from application build/deploy.

Exit condition: Local and remote migration state is understood; no migration runs implicitly during a Cloudflare deployment.

Gate 2 - Explicit account and property provisioning

- Decide which real user should be the platform administrator.
- Confirm identity UUID and role before any privilege change.
- Provision admin access through a secure, reviewed process.
- Decide who is the actual owner/host/co-host for Yash Home Stay and Wonderway Retreat.
- Assign only after confirming user/property IDs and authority.
- Test that ordinary users cannot self-promote or access unrelated properties.

Exit condition: A real admin and authorised host accounts can exercise only their intended permissions. No identity is auto-promoted.

Gate 3 - End-to-end stays operations

- Test guest sign-up, login, email verification and session lifecycle.
- Test booking request submission, date validation, capacity and server-side pricing.
- Test overlapping concurrent requests, back-to-back stays, blocks, accept/decline and failure recovery.
- Test host workspace with assigned host and guest trips with separate guest identity.
- Confirm all prices, amenities, availability and settlement language against the live properties.

Exit condition: A real test booking can complete from guest request to authorised host decision without a race condition or data leak.

Gate 4 - Products and seller operations

- Complete seller application review.
- Test seller-specific workspace permissions.
- Create real product records and moderation lifecycle.
- Validate provenance/claims and product imagery.
- Keep no-inventory states clear and honest.

Exit condition: At least one genuinely onboarded and approved seller/product exists before the catalogue is represented as a live marketplace.

Gate 5 - Productions and Community operations

- Apply the approved pending schema migration when authorised.
- Test creator applications, proposals, ownership, reviewer workflow and audit log.

- Verify real documentary assets, playback and any access restriction.
- Test community applications, consent, story moderation and participant status.
- Label initiatives as proposed, recruiting, active or completed based on real status evidence.

Exit condition: Submissions persist to production correctly and only approved/consented content is published.

Gate 6 - Legal, privacy and operations readiness

- Prepare accurate privacy notice and user terms.
- Prepare booking and cancellation terms that match request-based, pay-at-property operations.
- Prepare host/seller agreements before activating commercial relationships.
- Set consent and document-version capture after approved documents exist.
- Review retention/deletion processes, contact channels, backup/restore, incident response and support ownership.
- Obtain appropriate qualified Indian legal/accounting advice for the actual activities and entity structure.

Exit condition: The public site accurately describes the real service and the operator is ready to handle user data and customer issues.

Gate 7 - Final owner-controlled deployment

- Run all build/test commands against final source.
- Verify the generated OpenNext Worker and assets.
- Confirm Cloudflare Workers Builds root/build/deploy settings.
- Confirm Supabase Auth site URL and redirect URLs.
- Owner reviews and pushes code to the connected Git repository.
- Cloudflare Workers Builds deploys from Git.
- Owner performs a production smoke test on https://uttarkunth.com.

Exit condition: The owner authorises Git push; deployment is performed by the connected Cloudflare pipeline, not by an AI agent.

---

14. Google AI Studio working instructions

This section is intended to be copied into Google AI Studio as the initial project instruction after attaching this architecture document and providing the source code through a supported import method.

14.1 Project context instruction

You are helping me continue the existing Uttarkunth platform. Read the attached Uttarkunth Master Ecosystem Architecture & AI Studio Context Document first, then inspect the actual source code and configuration that I provide.

The architecture document is contextual guidance, not proof that every listed feature is implemented or operational. The source code is the authority for frontend/backend implementation; the live Supabase project is the authority for production database state; the Cloudflare dashboard and wrangler.jsonc are the authority for deployment configuration. Identify discrepancies instead of guessing.

Uttarkunth is a parent brand with four pillars: Stays, Products, Productions and Community. Preserve the existing visual identity, official logo, genuine photos and existing media. Do not replace the current design or assets without an explicit request.

The reported stack is Next.js 14 App Router, React 18, TypeScript, Supabase and Cloudflare Workers using OpenNext. Do not migrate frameworks or introduce paid infrastructure without first explaining why and obtaining approval.

Critical boundaries:
- Do not run production Supabase DDL or DML unless I explicitly authorise the exact change.
- Do not apply the pending Phase 2G/2H migration without my explicit approval.
- Do not reapply the three reported production migrations: 20261103114500, 20261103114530 and 20261103114600.
- Do not create an administrator or assign hosts automatically.
- Do not grant admin based on email address, profile metadata or signup.
- Do not invent products, sellers, community outcomes, video assets, reviews, prices or legal approvals.
- Do not expose or request the Supabase service-role key or other secrets.

- Do not initialise, commit, push or change Git remotes without my explicit instruction.
- Do not run wrangler login or wrangler deploy. I personally push code to GitHub; Cloudflare Workers Builds handles deployment.

First, inspect the project and report:
1. Exact framework/dependency versions and project structure.
2. Full route inventory and which routes are actually implemented.
3. Supabase clients, API routes, database types and migrations.
4. Which features depend on the pending Phase 2G/2H migration.
5. Authentication, RLS, role checks and server/client trust boundaries.
6. Cloudflare/OpenNext configuration and required environment variables.
7. Tests/build commands and any failing or missing checks.
8. Differences between source code, this architecture document and reported production state.

Do not start a large rewrite immediately. First produce a gap report and prioritised plan. When asked to implement a change, make the smallest safe change, preserve existing working features and design, run relevant tests, and provide exact file changes and results. Never claim a test, deployment, database migration or integration succeeded unless you actually ran and verified it.

14.2 How to use this document with source code

This document explains the system; it does not contain the complete application source. Upload or import the actual project code separately using the method available in the AI Studio mode/account you are using. If the project is already in GitHub, AI Studio's Build mode documentation describes an **Import from GitHub** workflow. Otherwise, provide the relevant source files through the interface's supported file/folder import option. Avoid uploading .env files, service-role keys, Cloudflare tokens or production database credentials.

For a large codebase, ask the AI to inspect the repository in stages: project/configuration first, then routes/components, then API and Supabase integration, then migrations/tests. The AI should produce a file inventory and dependency map before editing.

---

15. Final owner checklist

Before sharing code with an AI tool

- ☐ Attach this architecture document.
- ☐ Provide the real source project separately using a supported import method.
- ☐ Exclude all secret environment files and credentials.
- ☐ Provide the latest handoff report and any current failing logs if relevant.
- ☐ Ask the AI to inspect and report before editing.

Before Git upload

- ☐ Confirm repository path, branch and remote.
- ☐ Diagnose the previous Git status exit-code 1.
- ☐ Inspect git status --short and all diffs.
- ☐ Check .gitignore and scan for secrets.
- ☐ Reconcile local migrations with remote history.
- ☐ Keep the pending Phase 2G/2H migration unapplied until authorised.
- ☐ Re-run npm test.
- ☐ Re-run node scripts/test-isolated-pg.mjs.
- ☐ Re-run npx tsc --noEmit.
- ☐ Re-run npm run lint.
- ☐ Re-run npm run build.
- ☐ Re-run npm run build:worker.
- ☐ Confirm fresh .open-next/worker.js and .open-next/assets exist.
- ☐ Review Cloudflare Workers Builds settings and Supabase Auth redirects.

- ☐ Owner reviews and performs the Git push.

After Cloudflare deploy

- ☐ Confirm homepage, brand assets and mobile navigation.
- ☐ Confirm stays listing and both canonical property detail pages.
- ☐ Verify live prices/availability against actual Supabase data.
- ☐ Test guest authentication and booking request with a dedicated test account.
- ☐ Confirm host/admin functions are correctly gated until provisioned.
- ☐ Confirm products show honest Coming Soon/empty states.
- ☐ Test actual documentary playback, not just page loading.
- ☐ Test community pages and ensure proposed initiatives are not misrepresented as operating projects.
- ☐ Test universal search and notifications for data leakage.
- ☐ Check error handling, logs, redirects and SEO metadata.

---

16. Glossary and source-of-truth rules

Term	Meaning
RLS	PostgreSQL Row Level Security; restricts which rows a database role/session may access
RPC	A database function called by the application for a narrowly scoped operation
OpenNext	Adapter/tooling that packages Next.js for supported runtimes such as Cloudflare Workers
Worker	Cloudflare's JavaScript runtime unit serving the deployed application
host_id	Property ownership/primary host reference used by the reported property model
Pending migration	SQL file in the local repository that has not been applied to production
Canonical property	The intended primary property record used by the platform, not a duplicate or placeholder
Production-ready	A state proven by end-to-end tests, production configuration and operational readiness; not merely a successful compile/build

Source-of-truth hierarchy

When information conflicts, use this order:

1. **Actual current production database** for live records, policies, functions and migration history.
2. **Actual current repository/source code** for implemented routes, components, APIs, types and local migration files.
3. **Actual Cloudflare project configuration** for deployment settings and deployed version.
4. **Latest owner-approved written decisions** for business rules, branding and permissions.
5. **Antigravity reports and this document** as summaries to be verified, not as substitutes for the systems above.

Never silently reconcile conflicting information by guessing. Report the discrepancy, its impact and the safest resolution.

End of document
Uttarkunth - Learn by Doing • Grow by Serving 💜